



Specifica Tecnica

The Bitless (Gruppo 18):

Lorenzo Battistella (2103116), Edoardo De Piccoli (2101055), Davide Facco (2087852),
Anna Marini (2110986), Andrea Menegaldo (2116426), Samuele Vendramin (2111934),
Simone Zecchinato (2113189)

Informazioni documento			
Versione	Data	Stato	Destinatari
0.4.0	2026-07-27	Stesura	The Bitless, prof. T. Vardanega, prof. R. Cardin, Zucchetti

Contatti: thebitless.swe@gmail.com

Registro delle modifiche					
Versione	Data	Autore	Verificatore	Approvatore	Descrizione
0.4.0	2026-07-27	A. Marini	A. Menegaldo	-	Integrazione _G continua e analisi statica del backend, tracciamento dei requisiti _G nella sezione Tecnologie, riorganizzazione della struttura delle sezioni Architettura, Flusso dei dati e Tracciamento
0.3.0	2026-07-27	A. Marini	A. Menegaldo	-	Stesura della sezione Tecnologie
0.2.0	2026-07-27	A. Marini	A. Menegaldo	-	Riorganizzazione della struttura del documento e stesura della sezione Introduzione
0.1.0	2026-07-18	A. Menegaldo	A. Marini	-	Struttura del documento #1

Indice

1	Introduzione	4
1.1	Scopo del documento	4
1.2	Scopo del prodotto	4
1.3	Glossario	5
1.4	Riferimenti	5
1.4.1	Riferimenti normativi	5
1.4.2	Riferimenti informativi	5
2	Tecnologie	8
2.1	Criteri di scelta	8
2.2	Frontend	9
2.3	Backend	13
2.4	Persistenza delle note	16
2.5	Tecnologie di test e analisi	18
2.5.1	Analisi statica	18
2.5.2	Analisi dinamica	19
2.6	Infrastruttura	21
3	Architettura	24
3.1	Vista d'insieme	24
3.2	Architettura di deployment	24
3.2.1	Confronto con l'architettura a microservizi	24
3.2.2	Confronto con l'architettura monolitica	24
3.3	Architettura del frontend	24
3.3.1	Scomposizione in componenti	24
3.3.2	Gestione dello stato applicativo	24
3.4	Architettura del backend	24
3.4.1	Porte e adattatori	24
3.4.2	Scomposizione in moduli	24
3.5	Design pattern adottati	24
3.6	Idiomi e convenzioni di codifica	24
3.6.1	Generatori asincroni e propagazione dell'annullamento	24
3.6.2	Tipizzazione dei confini fra i livelli	24
3.6.3	Denominazione, struttura dei file e gestione degli errori	24
4	Flusso dei dati	24
4.1	Generazione assistita da LLM_G con risposta in streaming	24
4.2	Annullamento di un'operazione in corso	24
4.3	Generazione a partire da un collegamento	24
4.4	Salvataggio e caricamento di una nota	24
4.5	Propagazione e presentazione degli errori	24
5	Tracciamento dei requisiti$_G$	24
5.1	Criteri di tracciamento	24
5.2	Requisiti $_G$ funzionali	24
5.3	Requisiti $_G$ di qualità	24
5.4	Requisiti $_G$ di vincolo $_{GG}$	24
5.5	Grafici riassuntivi	24

Elenco delle tabelle

1	Tecnologie adottate per il frontend _G	10
2	Tecnologie adottate per il backend	13
3	Strumenti di analisi statica	18
4	Strumenti di analisi dinamica	19
5	Tecnologie di infrastruttura	21

1 Introduzione

1.1 Scopo del documento

Il presente documento descrive l'architettura del sistema_{GG} **Second Brain**, prodotto realizzato dal gruppo **The Bitless** in risposta al capitolato_{GG} **C6** proposto da **Zucchetti S.p.A.**. Mentre l'Analisi dei Requisiti_{GG} stabilisce *che cosa* il sistema_{GG} deve fare, la Specifica Tecnica stabilisce *come* il sistema_{GG} è costruito per farlo.

Nello specifico, il documento persegue i seguenti obiettivi:

- **Motivazione delle scelte tecnologiche:** espone le tecnologie adottate per ciascun livello del prodotto, i bisogni tecnici che ne hanno determinato la selezione e le alternative valutate e scartate, con le relative motivazioni.
- **Descrizione dell'architettura:** illustra la scomposizione del sistema_{GG} nelle sue componenti, le responsabilità attribuite a ciascuna di esse e le interazioni che ne regolano la collaborazione, sia in termini logici sia in termini di distribuzione delle parti in esecuzione.
- **Documentazione dei design pattern:** riporta i pattern architetturali e di progettazione adottati, motivandone l'impiego rispetto al problema specifico che risolvono all'interno del prodotto.
- **Tracciamento dei requisiti_G:** correla le componenti architetturali ai requisiti_{GG} individuati nell'Analisi dei Requisiti_{GG}, così da rendere verificabile la copertura del perimetro funzionale concordato con la proponente_{GG}.

Il documento costituisce il riferimento progettuale per le successive fasi di realizzazione e di manutenzione del prodotto: ogni intervento sul codice deve risultare coerente con quanto qui descritto e, qualora introduca una variazione architetturale, deve essere preceduto dall'aggiornamento della presente specifica.

La Specifica Tecnica dà inoltre continuità al **Proof of Concept_G** sviluppato per la **Requirements and Technology Baseline_G**: le tecnologie e le soluzioni architetturali qui documentate sono quelle sperimentate e validate in quella sede, ora consolidate e portate al livello di dettaglio necessario allo sviluppo_{GG} del prodotto completo.

Come gli altri documenti del gruppo, la stesura segue un approccio **incrementale_G**: il documento viene raffinato a ogni iterazione, in modo da riflettere costantemente lo stato effettivo del prodotto.

1.2 Scopo del prodotto

Il capitolato_{GG} **C6** di **Zucchetti S.p.A.** richiede la realizzazione di **Second Brain**, un'applicazione web per la scrittura e la gestione di note in formato **Markdown_G**, assistita da modelli linguistici di grandi dimensioni (**LLM_G**).

L'interfaccia si articola in due pannelli affiancati: a sinistra un editor in cui l'utente compone il testo nella sintassi **Markdown_G**, a destra un'anteprima che ne mostra il rendering aggiornato in tempo reale. Su questa base si innestano le funzioni assistite dall'**LLM_{GG}**, che operano sul testo selezionato o sull'intera nota: riassunto, riscrittura e adattamento del tono, correzione, traduzione multilingue, analisi critica secondo la tecnica dei **Sei Cappelli per Pensare_G** di Edward de Bono e generazione di testo secondo il paradigma del **Distant Writing_G**, in cui l'utente descrive sinteticamente il contenuto desiderato e delega al modello la stesura. Le note prodotte sono salvate e ricaricate sul file system locale dell'utente in formato **Markdown_G**.

Il valore del prodotto non risiede quindi nella sola redazione del testo, ma nel ruolo attivo assunto dal sistema_{GG}, che affianca l'utente nelle fasi di ideazione, revisione e riorganizzazione dei contenuti.

Per la trattazione completa del perimetro funzionale, dei casi d'uso_{GG} e della classificazione dei requisiti_{GG} in obbligatori, desiderabili e opzionali si rimanda all'*Analisi dei Requisiti*, di cui il presente documento non intende costituire una duplicazione.

1.3 Glossario

La creazione e lo sviluppo_{GG} di un sistema_{GG} software richiedono una complessa operazione di progettazione e analisi del dominio_{GG}, attività che deve necessariamente precedere la stesura del codice. Il gruppo si impegna a raccogliere e organizzare tali informazioni in modo che siano facilmente accessibili, al fine di favorire la massima asincronia ed efficienza nelle attività di progetto.

La principale fonte di ambiguità nello svolgimento delle attività è rappresentata dall'incomprensione dei termini tecnici o specialistici adottati. A tale scopo, la nomenclatura di riferimento viene raccolta nel *glossario*, un documento incrementale_{GG} volto a definire ogni termine rilevante per il dominio_{GG} del progetto.

Il gruppo si impegna a contrassegnare ogni occorrenza di un termine presente nel glossario mediante l'apposizione di una lettera **G** a pedice, come esemplificato di seguito:

parola_G

Al fine di garantire una corretta comprensione del dominio_{GG}, è fondamentale che ogni membro del gruppo consulti periodicamente il glossario, assicurando così il costante allineamento sulle definizioni e sui concetti chiave.

1.4 Riferimenti

1.4.1 Riferimenti normativi

- **Capitolato C6 - Second Brain:**
<https://www.math.unipd.it/~tullio/IS-1/2025/Progetto/C6.pdf>
Ultimo accesso: 20 maggio 2026.
- **Regolamento del Progetto Didattico:**
<https://www.math.unipd.it/~tullio/IS-1/2025/Dispense/PD1.pdf>
Ultimo accesso: 23 maggio 2026.
- **Norme di Progetto v1.0.0:**
https://thebitless.live/RTB/doc_interna/norme-di-progetto/norme-di-progetto.pdf
Ultimo accesso: 8 giugno 2026.

1.4.2 Riferimenti informativi

Documentazione di progetto

- **Glossario v1.0.0:**
https://thebitless.live/RTB/doc_interna/glossario/glossario.pdf
Ultimo accesso: 12 giugno 2026.

- **Analisi dei Requisiti v2.0.0:**
https://thebitless.live/RTB/doc_esterna/analisi-dei-requisiti/analisi-dei-requisiti.pdf
Ultimo accesso: 27 luglio 2026.
- **Verbale interno VI_2026-05-12 – Scelta delle tecnologie per il PoC_G:**
https://thebitless.live/RTB/doc_interna/verballi/VI_2026-05-12_quarto-sprint/VI_2026-05-12_quarto-sprint.pdf
Ultimo accesso: 27 luglio 2026.
- **Verbale interno VI_2026-05-20 – Progettazione del PoC_G:**
https://thebitless.live/RTB/doc_interna/verballi/VI_2026-05-20_progettazione-PoC/VI_2026-05-20_progettazione-PoC.pdf
Ultimo accesso: 27 luglio 2026.
- **Verbale esterno VE_2026-05-14 – Approvazione delle tecnologie da parte della proponente_G:**
https://thebitless.live/RTB/doc_esterna/verballi/VE_2026-05-14_AdR-scelta-tecnologie/VE_2026-05-14_AdR-scelta-tecnologie.pdf
Ultimo accesso: 27 luglio 2026.

Documentazione delle tecnologie adottate Documentazione ufficiale delle tecnologie descritte nella sezione 2. *Ultimo accesso* a tutte le risorse elencate di seguito: 27 luglio 2026.

- **TypeScript:** <https://www.typescriptlang.org/docs/>
- **React:** <https://react.dev/>
- **Zustand:** <https://zustand.docs.pmnd.rs/>
- **react-markdown:** <https://github.com/remarkjs/react-markdown>
- **Tailwind CSS:** <https://tailwindcss.com/docs>
- **shadcn/ui:** <https://ui.shadcn.com/>
- **Radix UI Primitives:** <https://www.radix-ui.com/primitives>
- **CodeMirror 6:** <https://codemirror.net/docs/>
- **Vite:** <https://vite.dev/>
- **Python 3.12:** <https://docs.python.org/3.12/>
- **FastAPI:** <https://fastapi.tiangolo.com/>
- **Pydantic:** <https://docs.pydantic.dev/>
- **httpx:** <https://www.python-httpx.org/>
- **Uvicorn:** <https://www.uvicorn.org/>
- **Tavily:** <https://docs.tavily.com/>
- **LiteLLM:** <https://docs.litellm.ai/>
- **Server-Sent Events (MDN):** https://developer.mozilla.org/en-US/docs/Web/API/Server-sent_events
- **File System Access API (MDN):** https://developer.mozilla.org/en-US/docs/Web/API/File_System_API
- **Specifica OpenAPI:** <https://spec.openapis.org/oas/latest.html>

- **openapi-typescript**: <https://openapi-ts.dev/>
- **ESLint**: <https://eslint.org/docs/latest/>
- **Ruff**: <https://docs.astral.sh/ruff/>
- **mypy**: <https://mypy.readthedocs.io/>
- **Vitest**: <https://vitest.dev/>
- **Copertura del codice in Vitest**: <https://vitest.dev/guide/coverage>
- **Testing Library**: <https://testing-library.com/docs/react-testing-library/intro/>
- **pytest**: <https://docs.pytest.org/>
- **Docker**: <https://docs.docker.com/>
- **Docker Compose**: <https://docs.docker.com/compose/>
- **GitHub Actions**: <https://docs.github.com/actions>
- **pnpm**: <https://pnpm.io/>
- **uv**: <https://docs.astral.sh/uv/>
- **Node.js – calendario di supporto delle versioni**: <https://github.com/nodejs/release>
- **File API – interfaccia File (MDN)**: <https://developer.mozilla.org/en-US/docs/Web/API/File>

Riferimenti metodologici

- **E. Gamma, R. Helm, R. Johnson, J. Vlissides**, *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley, 1994.
- **C4 model for visualising software architecture**:
<https://c4model.com/>
Ultimo accesso: 27 luglio 2026.

2 Tecnologie

2.1 Criteri di scelta

Le tecnologie descritte in questa sezione non sono state selezionate in astratto, sulla base della sola diffusione o popolarità, ma in risposta a bisogni tecnici precisi derivati dal capitolato_{GG} e dall'Analisi dei Requisiti_{GG}. Ciascuno dei bisogni elencati di seguito vincola una o più delle scelte successive, e costituisce il criterio rispetto al quale tali scelte vanno valutate.

1. Risposta progressiva. Un riassunto o una generazione di testo richiedono al modello linguistico un tempo di elaborazione dell'ordine dei secondi. Mantenere l'utente davanti a un semplice indicatore di attesa per tutta la durata dell'operazione non è accettabile sul piano dell'usabilità_G: il testo deve comparire progressivamente mentre viene prodotto (*streaming*). Questo bisogno determina la scelta di un livello server asincrono, di un client HTTP che supporti la lettura incrementale_G della risposta e di un protocollo di trasporto adatto al flusso unidirezionale di dati testuali. Il criterio discende dal requisito_G **R-109-F-De**, che impone un indicatore di avanzamento per l'intera durata delle operazioni con attesa percepibile, e da **R-79-F-De**, che richiede la possibilità di annullare un'elaborazione in corso: entrambi presuppongono che il server mantenga il controllo dell'operazione mentre questa procede, e non si limiti a restituirne l'esito finale.

2. Riservatezza delle credenziali dei fornitori esterni. L'accesso ai modelli linguistici avviene tramite una chiave di autenticazione fornita dalla proponente_{GG}, in attuazione del requisito di vincolo_{GG} **R-2-V-Ob**, che prescrive l'interfacciamento con i servizi LLM_{GG} tramite API_{GG}. Tale chiave non deve in alcun caso raggiungere il browser, dove sarebbe leggibile da chiunque ispezioni il traffico o il codice dell'applicazione. È quindi necessaria una componente server intermediaria che detenga la chiave e sia l'unico soggetto a comunicare con il fornitore; la medesima componente risolve anche il vincolo_{GG} **R-3-V-Ob** sulla *same-origin policy*, poiché è essa a esporre al browser un'origine controllata, configurata mediante la politica CORS.

Il criterio si estende a ogni fornitore esterno contattato dal prodotto, non al solo fornitore dei modelli. La funzione di generazione a partire da un collegamento (**R-74-F-De**) richiede l'estrazione del contenuto di una pagina web indicata dall'utente, operazione affidata a un secondo servizio esterno dotato di una propria chiave: anche questa è custodita dal backend e non transita dal browser. Va dichiarato, per trasparenza verso l'utente e verso la proponente_{GG}, **quali dati lasciano il perimetro del sistema_G**: verso il fornitore dei modelli linguistici transitano il testo sottoposto all'operazione — la porzione selezionata o l'intera nota — e le istruzioni costruite dall'applicazione; verso il servizio di estrazione transita il solo indirizzo fornito dall'utente. Nessuno dei due riceve dati identificativi dell'utente, che il prodotto non raccoglie.

Il confine è dato da ciò che l'utente sottopone, non dalla provenienza del testo: **nessun file è trasmesso automaticamente**. Una nota aperta da disco resta nel browser finché l'utente non ne affida il contenuto a un'operazione assistita; da quel momento esce dal perimetro come qualunque altro testo. Aprire un file non comporta dunque alcuna trasmissione, chiederne il riassunto sì: la differenza sta in un'azione che l'utente compie deliberatamente.

3. Rendering sicuro del Markdown_G. Il requisito_G **R-113-F-Ob** prevede una componente di rendering che trasformi il testo Markdown_G in una presentazione grafica formattata. Il testo restituito dal modello linguistico non è però contenuto fidato: è generato da un sistema_{GG} esterno a partire da input_{GG} che l'utente controlla solo in parte. Deve essere mostrato all'utente come HTML formattato senza che ciò apra la strada a vulnerabilità di tipo *cross-site scripting*

(XSS). La tecnologia di rendering deve quindi essere sicura per costruzione, non affidarsi a una sanificazione applicata a posteriori. Al medesimo criterio si riconduce **R-110-F-Ob**, che impone messaggi di errore in linguaggio naturale privi di dettagli tecnici: la gestione degli errori del modello non deve riversare nell'interfaccia contenuto proveniente dall'esterno.

4. Editor ricco ma stabile. L'editor deve offrire evidenziazione della sintassi, cronologia delle modifiche con annullamento e ripristino (**R-7-F-De**, **R-8-F-De**), e un comportamento accessibile da tastiera e da lettore di schermo; **R-111-F-Ob** ne prescrive la realizzazione come componente capace di ospitare testo su più righe. Deve inoltre mantenere invariati cursore e posizione di scorrimento mentre il pannello di anteprima si aggiorna in streaming, condizione che esclude soluzioni in cui la ricostruzione della vista coinvolge l'intera area di lavoro.

5. Manutenibilità_G con un gruppo di sette persone. Il prodotto è sviluppato in parallelo da sette persone con livelli di esperienza eterogenei. Ciò richiede codice tipizzato, in cui gli errori di interfaccia emergano in fase di compilazione anziché in esecuzione, confini netti fra i livelli dell'applicazione e verifica_{GG} automatica delle modifiche prima della loro integrazione_{GG}. Il criterio è vincolato da tre requisiti_{GG} di qualità: **R-4-Q-Ob** richiede che ogni pull request_{GG} sia sottoposta a revisione tra pari e superi i test_{GG} automatici prima dell'integrazione_{GG} nel ramo principale, **R-2-Q-Ob** richiede una copertura del codice_G conforme alle soglie del Piano di Qualifica_G, **R-10-Q-Ob** impone il rispetto delle Norme di Progetto_{GG}. Serve quindi un'infrastruttura di *Continuous Integration_G* che applichi questi controlli in modo automatico e uniforme sui due livelli.

6. Perimetro tecnologico ammesso dal capitolato_G. Il requisito di vincolo_{GG} **R-7-V-Op** stabilisce che la componente server-side, se realizzata, possa essere sviluppata utilizzando **Java** o **Python**. La scelta di Python ricade quindi all'interno del perimetro tecnologico ammesso dalla proponente_{GG}. Sul versante client, **R-1-V-Ob** circoscrive il perimetro alle Tecnologie Web_{GG} e ne fissa gli ambienti di esecuzione di riferimento: le versioni recenti di **Google Chrome**, **Mozilla Firefox** e **Microsoft Edge**. Questo vincolo_{GG} ha conseguenza diretta sulla strategia di persistenza descritta nella sezione 2.4, poiché le tre famiglie di browser non offrono le medesime interfacce di accesso al file system.

Filosofia trasversale. Le scelte che seguono rispondono a un unico criterio di fondo: **l'equilibrio fra controllo e velocità di sviluppo_G**. Il gruppo ha preferito tecnologie che lasciano visibile e modificabile ciò che accade, evitando tanto le soluzioni che nascondono il comportamento dietro astrazioni ampie e opinate, quanto quelle che impongono di riscrivere da zero funzionalità già disponibili. A parità di adeguatezza tecnica, la preferenza è andata agli ecosistemi maturi, con documentazione estesa e ampia disponibilità di risorse: fattore determinante per un gruppo che affronta gran parte di queste tecnologie per la prima volta.

2.2 Frontend

Il frontend_{GG} è un'applicazione a pagina singola eseguita interamente nel browser. Le tecnologie sono presentate nell'ordine con cui si sovrappongono: il linguaggio, la libreria che struttura la vista, l'ecosistema di librerie che ne realizza le funzioni specifiche e, infine, gli strumenti che ne governano la costruzione.

Tabella 1: Tecnologie adottate per il frontend_G

Tecnologia	Versione	Descrizione e ruolo nel progetto
TypeScript	6.0.3	Linguaggio di sviluppo _{GG} dell'intero frontend _{GG} . Estende JavaScript con un sistema _G di tipi statici che viene rimosso in fase di compilazione (<i>type erasure</i>): i tipi non esistono a tempo di esecuzione e non introducono alcun costo prestazionale. Il loro contributo è spostare in compilazione errori che si manifesterebbero altrimenti in produzione, in particolare sulle interfacce fra componenti. La validazione _G dei dati provenienti dall'esterno resta a carico del backend, poiché i tipi non sopravvivono alla compilazione e non possono verificare nulla a tempo di esecuzione.
React	19.2.7	Libreria per la costruzione dell'interfaccia, organizzata a componenti. Editor, anteprima, barra degli strumenti e pannelli delle funzioni assistite dall'LLM _{GG} sono componenti React distinti. Il meccanismo di <i>reconciliation</i> confronta la rappresentazione dell'interfaccia prima e dopo un cambiamento di stato e applica al DOM le sole modifiche effettivamente necessarie: durante lo streaming, in cui lo stato cambia decine di volte al secondo, questo evita la ricostruzione dell'intera pagina. L'applicazione non impiega alcun router : l'interazione si svolge in una sola vista, priva di navigazione fra pagine distinte, e l'introduzione di un instradamento lato client sarebbe una complessità non giustificata.
Zustand	5.0.14	Gestione dello stato applicativo condiviso fra i componenti (testo della nota, selezione corrente, testo in arrivo dallo streaming, stato di errore, modalità di visualizzazione). Espone selettori granulari, ciascuno dei quali osserva una singola porzione di stato: durante lo streaming la ridipintura interessa il solo pannello di anteprima, mentre l'editor resta immobile e conserva cursore e posizione di scorrimento.
react-markdown	10.1.0	Rendering del Markdown _G nel pannello di anteprima. Il testo viene analizzato e trasformato in un albero sintattico astratto, dal quale la libreria costruisce direttamente gli elementi React corrispondenti. In nessun punto della catena viene iniettato HTML grezzo nel documento: la libreria è quindi sicura per costruzione rispetto agli attacchi XSS, senza necessità di un passaggio di sanificazione. Il requisito è critico, poiché il testo da rendere proviene in larga parte dal modello linguistico.
remark-gfm	4.0.1	Estensione della pipeline di analisi con le costruzioni <i>GitHub Flavored Markdown</i> (tabelle, elenchi di attività, testo barrato, collegamenti automatici), attese dall'utenza di riferimento del prodotto. Le tabelle in particolare non appartengono al Markdown _G di base: l'estensione è la condizione perché i requisiti _{GG} R-34-F-De ... R-38-F-De (inserimento di una tabella e gestione di righe e colonne, UC44 ... UC48) trovino una resa nell'anteprima.

Tecnologia	Versione	Descrizione e ruolo nel progetto
remark-math rehype-katex KaTeX	6.0.0 7.0.1 0.17.0	Riconoscimento delle espressioni matematiche nel sorgente <code>Markdown_G</code> e loro composizione tipografica nell'anteprima. Realizzano i requisiti _{GG} R-28-F-De e R-29-F-De (inserimento e rimozione di una formula scientifica o matematica, UC35 e UC37), che senza un motore di composizione dedicato non sarebbero verificabili _G sul lato dell'anteprima. Operano come estensioni della stessa pipeline ad albero sintattico, preservandone la proprietà di non iniettare HTML grezzo.
rehype-highlight highlight.js	7.0.2 11.11.1	Evidenziazione della sintassi all'interno dei blocchi di codice mostrati nell'anteprima. Discende dai requisiti _{GG} R-26-F-De , R-27-F-De e R-119-F-De (inserimento, rimozione e modifica di un blocco di codice sorgente, UC32, UC34 e UC33), che presuppongono una resa del blocco di codice distinta dal testo ordinario.
Tailwind CSS	4.3.1	Sistema _G di stile <i>utility-first</i> : la presentazione è espressa mediante classi elementari applicate direttamente al marcatore, anziché tramite fogli di stile separati. Ne deriva un insieme chiuso di valori (spaziature, colori, dimensioni tipografiche) che mantiene l'aspetto uniforme fra le parti dell'interfaccia scritte da persone diverse. Il plugin <code>@tailwindcss/typography</code> (0.5.20) fornisce gli stili tipografici applicati al risultato del rendering <code>Markdown_G</code> .
shadcn/ui Radix UI	— 1.5.0	Insieme di componenti di interfaccia (pulsanti, aree di testo, finestre di dialogo, avvisi). <code>shadcn/ui</code> non è una dipendenza installata ma un catalogo da cui il codice sorgente dei componenti viene copiato nel progetto: resta quindi interamente leggibile e modificabile dal gruppo, senza vincolo _G di adozione verso un fornitore esterno. I componenti poggiano su Radix UI, che fornisce le primitive di comportamento accessibile — navigazione da tastiera, gestione del focus, attributi per i lettori di schermo — la cui realizzazione manuale corretta sarebbe onerosa e soggetta a errori.
CodeMirror 6	6.0.2	Editor di testo del pannello sinistro. Fornisce nativamente evidenziazione della sintassi <code>Markdown_G</code> (modulo <code>@codemirror/lang-markdown</code> 6.5.0), cronologia delle modifiche con annullamento e ripristino (<code>@codemirror/commands</code> 6.10.3) e supporto all'accessibilità, a fronte di un ingombro dell'ordine del centinaio di kilobyte. L'architettura modulare consente di includere le sole estensioni effettivamente utilizzate.
lucide-react	1.18.0	Insieme di icone vettoriali impiegate nella barra degli strumenti e nei comandi dell'interfaccia.
Vite	8.0.16	Strumento di sviluppo _{GG} e costruzione del frontend _{GG} . In sviluppo _{GG} serve i moduli ES nativamente al browser, senza ricostruire l'intero pacchetto a ogni modifica, e aggiorna i moduli modificati mantenendo lo stato dell'applicazione; in produzione genera il pacchetto ottimizzato. Costituisce inoltre la base di configurazione riutilizzata dall'infrastruttura di test _G descritta nella sezione 2.5.

Alternative considerate

- **Angular, Vue e Svelte al posto di React**

Il confronto documentato nel verbale interno del 12 maggio 2026 ha contrapposto React e Svelte. Svelte adotta un approccio a compilatore: elimina il Virtual DOM spostando in fase di compilazione il lavoro che React svolge in esecuzione, con prestazioni superiori e minore ingombro del pacchetto prodotto. Il suo ecosistema è però più giovane, con minore disponibilità di librerie mature per le esigenze specifiche del prodotto e minore reperibilità di risorse di apprendimento, fattore rilevante per un gruppo che affronta la tecnologia per la prima volta. Vue è a sua volta maturo, ma la via più diffusa per il rendering del Markdown_G nel suo ecosistema consiste nel produrre HTML e iniettarlo nel documento, esattamente il rischio_G che il criterio 3 impone di evitare. Angular offre un quadro strutturale completo e fortemente prescrittivo, sovradimensionato per un'applicazione a vista singola e con una curva di apprendimento incompatibile con i tempi del progetto. La determinante è che le librerie richieste — rendering Markdown_G sicuro, componenti accessibili, integrazione_{GG} dell'editor — individuano React come primo ambiente di destinazione.

- **Redux, Context API e Jotai al posto di Zustand**

Redux impone una quantità di codice di supporto (azioni, riduttori, selettori memorizzati) sproporzionata rispetto alla dimensione dello stato da gestire. La Context API_G, disponibile nativamente in React, non offre sottoscrizioni granulari: ogni variazione del valore del contesto provoca la ridipintura di tutti i componenti che vi accedono, comportamento penalizzante durante lo streaming, in cui lo stato cambia con frequenza elevata e ridipingere l'editor comporterebbe la perdita della posizione del cursore. Jotai adotta un modello ad atomi tecnicamente adeguato, ma richiede di scomporre lo stato in unità elementari fin dall'inizio: Zustand ottiene il medesimo risultato con un modello concettuale più semplice e una superficie di apprendimento minore.

- **marked e markdown-it al posto di react-markdown**

Entrambe le librerie convertono il Markdown_G in una stringa HTML, che dovrebbe poi essere inserita nel documento tramite un meccanismo di iniezione diretta. Ciò renderebbe obbligatorio un passaggio esplicito di sanificazione, la cui correttezza dipenderebbe dalla configurazione adottata e andrebbe verificata_G separatamente. react-markdown rende il problema inesistente per costruzione.

- **MUI e Chakra UI al posto di shadcn/ui**

Offrono un numero maggiore di componenti già pronti, ma impongono un linguaggio visivo marcato, la cui personalizzazione richiede di intervenire contro le impostazioni predefinite della libreria; inoltre il codice dei componenti resta all'interno della dipendenza, non modificabile. L'alternativa opposta, i CSS Modules, non aggiunge alcuna dipendenza ma non fornisce né un sistema_G di valori condiviso né componenti accessibili, che andrebbero realizzati integralmente dal gruppo.

- **Monaco e textarea nativa al posto di CodeMirror**

Monaco è l'editor impiegato in Visual Studio Code: è progettato per la scrittura di codice sorgente e la sua dotazione di funzioni — completamento automatico, analisi del linguaggio, riquadro di anteprima — eccede quanto richiesto dalla redazione di note, a fronte di un ingombro di oltre un ordine di grandezza superiore. L'elemento `textarea` nativo non comporta alcuna dipendenza, ma è privo di evidenziazione della sintassi e di una cronologia delle modifiche strutturata.

- **Webpack e Create React App al posto di Vite**

Ricostruiscono l'intero pacchetto a ogni modifica, con tempi di attesa in sviluppo_{GG} sensibilmente superiori, e richiedono una configurazione più estesa e frammentata. Create React App non è

inoltre più oggetto di manutenzione attiva.

2.3 Backend

Il backend è una componente server che espone alcune API_{GG} tramite HTTP. Le sue responsabilità sono quattro:

1. custodire le chiavi di accesso ai fornitori esterni, così che non raggiungano mai il browser (criterio 2);
2. comporre le istruzioni da inviare al modello linguistico;
3. ritrasmettere al browser il testo prodotto man mano che diviene disponibile (criterio 1);
4. acquisire il contenuto testuale di una pagina web indicata dall'utente, quando la generazione avviene a partire da un collegamento.

La quarta responsabilità discende dal requisito_G **R-74-F-De** (UC67.2 e UC67.5) e non è riducibile alle altre tre: comporta un secondo fornitore esterno, distinto da quello dei modelli, e per questo è trattata separatamente nel paragrafo *Estrazione del contenuto delle pagine web*. Come per il frontend_{GG}, le tecnologie sono presentate nell'ordine linguaggio, framework_{GG}, librerie, strumenti.

Tabella 2: Tecnologie adottate per il backend

Tecnologia	Versione	Descrizione e ruolo nel progetto
Python	3.12	Linguaggio di sviluppo _{GG} del backend. Rientra nel perimetro tecnologico ammesso dal requisito _G R-7-V-Op. Dispone del supporto nativo alla programmazione asincrona (async/await), condizione necessaria per gestire attese di rete prolungate senza bloccare il processo, e dell'ecosistema più esteso per l'integrazione _{GG} con i modelli linguistici.
FastAPI	0.136.1	Framework _{GG} web su cui è costruita l' API_{GG} . È asincrono per impostazione, requisito imprescindibile per lo streaming; deriva la validazione _G dei corpi delle richieste direttamente dalle annotazioni di tipo, senza codice di controllo scritto a mano; genera automaticamente lo schema OpenAPI dell'interfaccia esposta. La risposta in streaming è realizzata mediante StreamingResponse , che consuma un generatore asincrono e gestisce la trasmissione a blocchi lungo la connessione HTTP.
Pydantic	2.13.4	Validazione _G e serializzazione dei dati in ingresso e in uscita. Gli schemi Pydantic costituiscono la fonte unica di verità dei contratti dell' API_{GG} : da essi FastAPI deriva lo schema OpenAPI e, da quest'ultimo, vengono generati i tipi TypeScript utilizzati dal frontend _{GG} (si veda la sezione 2.6). Il contratto è definito quindi una volta sola, in un solo punto del sistema _{GG} .
pydantic-settings	2.14.1	Lettura e validazione _G della configurazione applicativa (indirizzo del gateway dei modelli, identificativo del modello, chiave di accesso, origini ammesse dalla politica CORS) a partire dalle variabili d'ambiente. Le impostazioni sono esposte tramite un'unica istanza per processo.

Tecnologia	Versione	Descrizione e ruolo nel progetto
httplib	0.28.1	Client HTTP asincrono impiegato per la comunicazione verso il provider dei modelli linguistici. Supporta la lettura della risposta a blocchi man mano che arriva, senza attenderne il completamento, e propaga l'annullamento lungo l'intera catena: se l'utente interrompe la generazione, la chiusura si trasmette dal browser al backend e da questo alla connessione verso il provider, senza che rimangano richieste attive prive di destinatario.
tavily-python	0.7.26	Estrazione del contenuto testuale delle pagine web nella funzione di generazione a partire da un collegamento (requisito _G R-74-F-De , UC67.2 e UC67.5). Restituisce il solo testo della pagina, già ripulito dal marcatore e dagli elementi accessori, e ne consente il troncamento a una lunghezza massima prima dell'invio al modello. È un client verso un servizio esterno: si veda il paragrafo <i>Estrazione del contenuto delle pagine web</i> per il perimetro dei dati trasmessi e per l'alternativa scartata.
Uvicorn	0.47.0	Server ASGI sul quale l'applicazione FastAPI è posta in esecuzione. Costituisce l'implementazione dell'interfaccia asincrona fra il server HTTP e il codice applicativo.
Server-Sent Events	—	Protocollo di trasporto dei blocchi testuali dal backend al browser. È un formato testuale unidirezionale che viaggia su una normale connessione HTTP mantenuta aperta, in cui ogni evento è una riga con prefisso data: e la fine del flusso è segnalata da un marcatore convenzionale. La direzione unica — dal server al client — corrisponde esattamente alla natura del problema.

Accesso ai modelli linguistici. L'accesso ai modelli avviene attraverso un **gateway LiteLLM**, componente esterna che espone un'interfaccia compatibile con quella di OpenAI e uniforma l'accesso a fornitori diversi. Poiché si tratta di un servizio contattato via HTTP e non di una libreria installata nel progetto, non figura fra le dipendenze del backend e non è associato a una versione dichiarata nei file di progetto.

La comunicazione con il gateway è confinata dietro l'astrazione **LLMClient**, una classe astratta che dichiara la sola operazione di produzione incrementale_G del testo. L'implementazione concreta che dialoga con LiteLLM — interpretandone il formato di risposta e traducendone gli errori — ne è l'unico realizzatore, secondo il pattern Adapter descritto nella sezione dedicata all'architettura. **Né i router né la logica di dominio_G invocano direttamente LiteLLM:** conoscono unicamente l'interfaccia astratta. La sostituzione del gateway con un fornitore diverso comporta pertanto la scrittura di una nuova implementazione dell'interfaccia, e nessuna modifica al resto del backend.

Estrazione del contenuto delle pagine web. Il requisito_G **R-74-F-De** consente all'utente di indicare un URL come fonte di contesto per la generazione automatica di testo (UC67.2). Poiché il modello linguistico non accede alla rete, il contenuto della pagina deve essere acquisito dal prodotto e passato al modello come parte delle istruzioni. L'operazione è affidata a **Tavily**, servizio esterno che restituisce il testo di una pagina già ripulito dal marcatore, dalla navigazione e dagli elementi accessori.

Si tratta del **secondo fornitore esterno** del sistema_{GG}, dotato di una propria chiave di accesso. Coerentemente con il criterio 2, la chiave è custodita dal backend e non raggiunge il browser; verso il servizio transita esclusivamente l'indirizzo indicato dall'utente, non il testo della

nota. Il contenuto restituito è troncato a una lunghezza massima prima di essere inoltrato al modello, così da mantenere prevedibile la dimensione delle istruzioni.

L'alternativa considerata è l'**estrazione lato server con una libreria locale: trafilatura** e le realizzazioni Python dell'algoritmo *Readability* scaricano la pagina con il client HTTP già presente nel progetto e ne isolano il contenuto principale senza coinvolgere alcun terzo. Eliminerebbero un fornitore, una chiave e un punto di uscita dei dati — vantaggio non trascurabile — ma sposterebbero sul gruppo la qualità dell'estrazione, che su pagine costruite dinamicamente tramite JavaScript risulta sensibilmente inferiore, e richiederebbero di gestire in proprio reindirizzamenti, codifiche e tempi di risposta dei siti contattati. Poiché R-74-F-De è desiderabile e non obbligatorio, il gruppo ha preferito la soluzione che ne rende prevedibile il completamento, accettando la dipendenza esterna.

Rispetto all'accesso ai modelli linguistici c'è però un'**asimmetria**, che va detta per non attribuire all'estrazione una sostituibilità che oggi non ha. L'accesso al gateway passa dall'interfaccia astratta `LLMClient` e arriva ai router per iniezione della dipendenza; l'estrazione, invece, è affidata a una funzione che costruisce e invoca direttamente il client del fornitore, ed è il router a chiamarla. **Per l'estrazione non esiste quindi una porta del dominio_G analoga a `LLMClient`:** cambiare fornitore richiederebbe oggi di modificare il modulo di estrazione, non di scrivere soltanto un nuovo adattatore. Introdurre quella porta è un intervento circoscritto — un'interfaccia astratta, un adattatore che racchiuda il fornitore attuale e l'iniezione della dipendenza nel router — e allineerebbe l'estrazione al trattamento già riservato all'accesso ai modelli. Finché non avviene, cambiare fornitore di estrazione resta più oneroso che cambiare gateway.

Alternative considerate

- **Node.js con TypeScript e Go al posto di Python**

Entrambi sarebbero tecnicamente adeguati: Node.js consentirebbe di adottare un unico linguaggio sui due livelli, Go offrirebbe prestazioni superiori nella gestione di connessioni concorrenti prolungate. Nessuno dei due rientra però nel perimetro tecnologico ammesso dal requisito_G R-7-V-Op, che indica Java e Python. Java rientra nel perimetro ed è stato valutato, ma è stato scartato per la minore immediatezza del suo ecosistema nell'integrazione_{GG} con i modelli linguistici e per la maggiore verbosità rispetto alla dimensione contenuta del backend, che espone poche interfacce e non contiene logica di dominio_{GG} estesa.

- **Flask e Django al posto di FastAPI**

Flask è sincrono per impostazione: il supporto allo streaming asincrono richiederebbe estensioni aggiuntive e una configurazione non prevista dal suo modello di base, oltre a un livello di validazione_G scritto separatamente. Django è orientato ad applicazioni complete, con mappatore relazionale a oggetti, sistema_G di autenticazione e pannello di amministrazione integrati: componenti che il prodotto non utilizza, dato che non prevede persistenza su base di dati (si veda la sezione 2.4), e che ne renderebbero l'adozione sproporzionata.

- **requests e aiohttp al posto di httpx**

La libreria `requests` è lo standard di fatto per le richieste HTTP in Python, ma è esclusivamente sincrona: bloccherebbe il processo per l'intera durata della risposta del modello, vanificando la scelta di un framework_{GG} asincrono. La libreria `aiohttp` è asincrona e supporta lo streaming, ma espone un'ergonomia meno lineare nella gestione della chiusura anticipata delle connessioni, aspetto centrale per la funzione di annullamento della generazione.

- **SDK diretto del fornitore e LangChain al posto del gateway**

L'impiego dell'SDK proprietario di un singolo fornitore legherebbe il codice a quel fornitore, in

un ambito in cui modelli e condizioni di accesso cambiano con frequenza elevata. LangChain, all'estremo opposto, introduce un livello di astrazione ampio e fortemente opinato — catene, agenti, memoria, strumenti — di cui il prodotto non utilizzerebbe che una frazione minima, a fronte di un aumento sensibile della superficie di dipendenza e di una minore prevedibilità del comportamento. Il gateway, unito all'interfaccia **LLMClient**, offre l'indipendenza dal fornitore senza alcuno dei due inconvenienti.

- **WebSocket e polling al posto dei Server-Sent Events**

I WebSocket stabiliscono un canale bidirezionale, con un protocollo di negoziazione dedicato e la necessità di gestire esplicitamente riconessioni e stato della connessione: capacità superflue, poiché nel prodotto i dati scorrono in una sola direzione. Il *polling* periodico richiederebbe di accumulare il testo prodotto lato server in attesa di essere richiesto, introducendo latenza e complessità di gestione dello stato intermedio.

2.4 Persistenza delle note

Il gruppo ha deliberato di non realizzare la persistenza delle note su base di dati. Le note sono salvate e ricaricate esclusivamente sul **file system locale** della macchina dell'utente, per il tramite del browser.

Meccanismo adottato. Le operazioni di salvataggio e di caricamento sono realizzate con una strategia a due vie, selezionata a tempo di esecuzione in base alle capacità del browser ospite.

Quando il browser espone la **File System Access API** — verificata mediante la presenza dei metodi `showOpenFilePicker` e `showSaveFilePicker` sull'oggetto globale della finestra — l'applicazione apre la finestra di selezione file nativa del sistema_{GG} operativo. Il caricamento ottiene un riferimento al file scelto e ne legge il contenuto testuale; il salvataggio ottiene un riferimento al percorso di destinazione e vi scrive un flusso di byte. L'annullamento da parte dell'utente è riconosciuto e trattato come esito legittimo, non come errore.

Quando l'interfaccia non è disponibile — condizione che ricorre su **Firefox** e **Safari**, che non la implementano — l'applicazione ricade su meccanismi supportati universalmente. La doppia via non è una precauzione generica: il requisito di vincolo_{GG} **R-1-V-Ob** include Firefox fra gli ambienti di esecuzione di riferimento, quindi una realizzazione fondata sulla sola File System Access API_G lascerebbe scoperto uno dei tre browser prescritti. In caricamento crea un elemento **input** di tipo file e ne legge il contenuto tramite un lettore di file; in salvataggio costruisce un oggetto **Blob** con il contenuto della nota, ne ricava un indirizzo temporaneo e lo assegna a un collegamento di scaricamento attivato per via programmatica, delegando al browser la scelta della destinazione secondo le impostazioni dell'utente. L'indirizzo temporaneo viene rilasciato al termine dell'operazione.

Il criterio di selezione è quindi la disponibilità effettiva dell'interfaccia nel browser in uso, non l'identificazione del browser stesso: l'applicazione non interroga l'agente utente, ma verifica_G direttamente la presenza dei metodi richiesti. La funzione resta pertanto disponibile su ogni browser, con la sola differenza che sui browser dotati dell'interfaccia nativa l'utente sceglie esplicitamente il percorso di destinazione, mentre sugli altri il file segue le impostazioni di scaricamento predefinite.

Formato dei file. Le note sono salvate in **Markdown_G**, con estensione `.md` e tipo MIME `text/markdown`; in caricamento sono accettate anche le estensioni `.txt`. La scelta del testo semplice attua il requisito di vincolo_{GG} **R-4-V-Ob**, che prescrive la memorizzazione delle note salvate localmente come file di testo. Il file contiene il solo testo della nota, senza intestazioni

o marcatori aggiuntivi: resta quindi leggibile e modificabile con qualsiasi altro strumento, e non introduce alcun vincolo_{GG} verso il prodotto.

Metadati della nota. I metadati che l'applicazione associa alla nota — identificativo, titolo, istante di creazione e istante di ultima modifica — non sono scritti nel file, che resta un documento Markdown_G puro. Il titolo è veicolato dal nome del file: in salvataggio il nome proposto deriva dal titolo della nota, in caricamento il titolo è ricavato dal nome del file privato dell'estensione.

Questa impostazione ha una conseguenza sul requisito_G **R-97-F-De** (UC84), che richiede la visualizzazione delle informazioni e dei metadati di una nota selezionata: un identificativo e un istante di creazione assegnati in memoria non sopravvivono alla chiusura della sessione, e ricaricando il file da disco andrebbero perduti. Trattandosi di un requisito_G **desiderabile** e non opzionale, la copertura non può essere rinviata.

Il recupero è ottenuto senza modificare il formato del file, sfruttando i metadati che il file system già espone. L'oggetto **File** restituito sia dalla File System Access API_G sia dall'elemento **input** di tipo file riporta gli attributi **name**, **lastModified** e **size**: all'apertura di una nota l'applicazione li impiega per popolare rispettivamente titolo, istante di ultima modifica e dimensione del contenuto. Il numero di caratteri e il numero di parole sono derivati dal testo caricato. Resta non ricostruibile il solo **istante di creazione**, che il file system espone in modo non uniforme fra le piattaforme e che le interfacce del browser non rendono disponibile: per le note ricaricate da disco tale informazione è presentata come non disponibile anziché con un valore convenzionale, così che l'interfaccia non affermi un dato che il prodotto non possiede. L'identificativo continua a essere assegnato per la durata della sessione, non avendo significato al di fuori di essa in assenza di persistenza server-side.

Motivazione dell'esclusione della base di dati. La persistenza server-side è prevista dal capitolato_{GG} come estensione facoltativa. I requisiti_{GG} funzionali che ne derivano sono classificati come opzionali nell'Analisi dei Requisiti_{GG} e riguardano quattro capacità distinte:

- **operazioni sull'archivio remoto:** R-83-F-Op (creazione di una nota nella base di dati, UC74.2), R-86-F-Op (caricamento, UC76.2), R-89-F-Op (salvataggio, UC78.2), R-92-F-Op (rimozione, UC80.2), R-95-F-Op (rinomina, UC82.2). Sono il nucleo di ciò che l'esclusione comporta: le quattro operazioni di base sulle note, più la rinomina;
- **ricerca e filtro:** R-98-F-Op (ricerca per titolo), R-99-F-Op (ricerca full-text), R-100-F-Op e R-101-F-Op (filtro per data di creazione e di ultima modifica);
- **autenticazione:** R-102-F-Op (accesso con credenziali), R-103-F-Op (gestione degli errori di autenticazione), R-108-F-Op (disconnessione);
- **gestione dell'account:** R-104-F-Op (registrazione), R-105-F-Op (unicità dello username), R-106-F-Op (conferma della password), R-107-F-Op (rimozione dell'account e dei dati associati).

Sono opzionali anche i requisiti_{GG} di vincolo_{GG} corrispondenti: **R-5-V-Op** (memorizzazione delle note in una base di dati raggiungibile dal sistema_{GG}), **R-6-V-Op** (componente server raggiungibile tramite chiamate HTTP) e **R-7-V-Op** (linguaggio della componente server-side). Si segnala che dei tre soltanto R-7-V-Op è già soddisfatto dall'architettura attuale, poiché la componente server esiste per le funzioni assistite da LLM_{GG} indipendentemente dalla persistenza; R-5-V-Op e R-6-V-Op restano scoperti insieme alla funzionalità che li condiziona.

Il gruppo ha scelto di concentrare le risorse della presente iterazione sul completamento dei requisiti_{GG} obbligatori e desiderabili, in particolare sulle funzionalità assistite da LLM_{GG}, che

costituiscono il nucleo del prodotto richiesto dalla proponente_{GG}. Lo stato di copertura di ciascun requisito_G è riportato nella sezione 5.

Va precisato l'effettivo margine di estensione, per non attribuire all'architettura una predisposizione che non possiede. Nel perimetro attuale la persistenza delle note risiede **interamente nel frontend_G**: le note non raggiungono in alcun momento il backend, e nel dominio_{GG} del backend non esiste una porta per le note in attesa di essere implementata. Introdurre la persistenza su base di dati comporterebbe quindi lavoro nuovo e non marginale: entità di dominio_{GG} per la nota e per l'utente, le porte corrispondenti, gli endpoint che le espongono, un adattatore verso il sistema_{GG} di gestione della base di dati e l'intero blocco di autenticazione richiesto da R-102-F-Op ... R-108-F-Op.

Ciò che l'architettura esagonale garantisce è un'altra proprietà, più circoscritta ma sufficiente: la persistenza sarebbe aggiunta **come nuova porta e nuovo adattatore**, senza perturbare il dominio_{GG} delle funzioni assistite da LLM_{GG} già realizzato. Il costo dell'estensione ricade sulle componenti nuove, non sulla riscrittura di quelle esistenti.

2.5 Tecnologie di test e analisi

Le attività di verifica_{GG} previste dalle Norme di Progetto_{GG} si articolano in analisi statica_G, condotta sul codice sorgente senza porlo in esecuzione, e analisi dinamica_G, condotta eseguendo il codice su casi di prova predisposti.

2.5.1 Analisi statica

L'analisi statica_G intercetta gli errori prima che il codice venga eseguito, spostandone il rilevamento dalla fase di prova a quella di scrittura.

Tabella 3: Strumenti di analisi statica

Tecnologia	Versione	Descrizione e ruolo nel progetto
Compilatore TypeScript	6.0.3	Verifica _{GG} della coerenza dei tipi sull'intero codice del frontend _{GG} . La compilazione è eseguita in modo esplicito come primo passo del comando di costruzione (<code>tsc -b && vite build</code>): un errore di tipo interrompe la costruzione e impedisce la produzione del pacchetto. La configurazione attiva inoltre la segnalazione di variabili e parametri dichiarati e non utilizzati, dei casi non terminati nei costrutti di selezione multipla e delle costruzioni non rimovibili per sola cancellazione dei tipi.
ESLint	10.5.0	Analisi statica _G del codice del frontend _{GG} rispetto a un insieme di regole configurate. Individua costrutti sintatticamente validi ma problematici, che il compilatore non segnala.
typescript-eslint	8.61.0	Estensione di ESLint alla sintassi TypeScript e alle regole specifiche del linguaggio.
eslint-plugin-react-hooks	7.1.1	Verifica _{GG} del rispetto delle regole di impiego degli <i>hook</i> di React, in particolare della completezza degli elenchi di dipendenze. Si tratta di una classe di errori che non produce alcun messaggio in esecuzione ma si manifesta come stato dell'interfaccia non aggiornato, di difficile diagnosi.
eslint-plugin-react-refresh	0.5.2	Verifica _{GG} che i moduli rispettino le condizioni necessarie all'aggiornamento a caldo dei componenti durante lo sviluppo _{GG} .

Tecnologia	Versione	Descrizione e ruolo nel progetto
Ruff	—	Analisi statica _G e formattazione automatica del codice del backend. Riunisce in un solo strumento le regole di più analizzatori dell'ecosistema Python (fra cui quelle di pyflakes , pycodestyle e isort), segnalando importazioni e variabili non utilizzate, ordinamento delle importazioni, costrutti sintatticamente validi ma problematici e violazioni delle convenzioni di stile. Il formattatore integrato riproduce lo stile di black , indicato dalle Norme di Progetto _{GG} per la formattazione del codice Python: la conformità è quindi mantenuta senza introdurre un secondo strumento. La configurazione risiede in pyproject.toml , accanto a quella delle dipendenze.
mypy	—	Verifica _{GG} statica delle annotazioni di tipo del backend, in modalità rigorosa. Controlla la coerenza fra le firme dichiarate e le chiamate effettive, la completezza delle annotazioni e la correttezza dei tipi dei generatori asincroni impiegati nello streaming — classe di errori che, in un linguaggio a tipizzazione dinamica, si manifesterebbe altrimenti solo in esecuzione e, nel caso dello streaming, a operazione già avviata. Porta il backend al medesimo livello di controllo che il compilatore TypeScript garantisce sul frontend _{GG} .

L'analisi statica_G è quindi applicata a **entrambi** i livelli del prodotto, come richiesto dalle Norme di Progetto_{GG} e, per il loro tramite, dal requisito_G di qualità **R-10-Q-Ob**. La simmetria fra i due livelli è voluta: al compilatore TypeScript ed ESLint sul frontend_{GG} corrispondono mypy e Ruff sul backend, con la medesima ripartizione fra verifica_{GG} dei tipi e verifica_{GG} delle regole di scrittura.

Si osserva che la validazione_G operata dagli schemi **Pydantic** non appartiene a questa categoria e non ne costituisce un sostituto: Pydantic controlla i *dati* che attraversano i confini dell'applicazione a tempo di esecuzione, mentre l'analisi statica_G controlla il *codice* senza eseguirlo. Le due verifiche_G intercettano difetti disgiunti — un corpo di richiesta malformato nel primo caso, un'incoerenza fra firme di funzione nel secondo — e sono pertanto complementari.

2.5.2 Analisi dinamica

L'analisi dinamica_G è organizzata su test_{GG} di unità e di integrazione_{GG} dei due livelli, eseguiti separatamente dalle rispettive catene di strumenti.

Tabella 4: Strumenti di analisi dinamica

Tecnologia	Versione	Descrizione e ruolo nel progetto
Vitest	4.1.8	Esecutore dei test _{GG} di unità e di integrazione _{GG} del frontend _{GG} . Riutilizza integralmente la configurazione di Vite — risoluzione dei moduli, alias dei percorsi, trasformazione di TypeScript e JSX — evitando la manutenzione di una seconda catena di strumenti allineata alla prima.

Tecnologia	Versione	Descrizione e ruolo nel progetto
@vitest/coverage-v8	—	Misurazione della copertura del codice _G raggiunta dai test _{GG} del frontend _{GG} . Si appoggia allo strumento di copertura nativo del motore V8, senza richiedere una strumentazione del codice sorgente. Le soglie minime sono dichiarate nella configurazione di Vitest, così che il superamento sia verificato _G dall'esecutore stesso e non affidato alla lettura del rapporto. È la controparte di pytest-cov sul versante frontend _{GG} : senza di essa il Piano di Qualifica _G non potrebbe riportare una metrica _G di copertura su questo livello, come richiesto da R-2-Q-Ob .
jsdom	29.1.1	Realizzazione del DOM in ambiente Node.js, che consente di eseguire i test _{GG} dei componenti senza avviare un browser.
Testing Library (React)	16.3.2	Interrogazione dei componenti resi attraverso ruoli, etichette e testo visibile, così come vi accederebbe una persona che utilizza il prodotto, anziché attraverso la struttura interna del marcatore. I test _{GG} restano quindi validi a fronte di riorganizzazioni interne dei componenti e verificano indirettamente l'accessibilità dell'interfaccia.
jest-dom user-event	6.9.1 14.6.1	Asserzioni specifiche per gli elementi del DOM e simulazione delle interazioni dell'utente (digitazione, selezione, attivazione di comandi) con la sequenza di eventi effettivamente prodotta da un browser.
pytest	9.0.3	Esecutore dei test _{GG} del backend. Il sistema _G delle <i>fixture</i> consente di comporre e riutilizzare le precondizioni dei test _{GG} in modo esplicito.
pytest-asyncio	1.4.0	Esecuzione dei test _{GG} su funzioni asincrone, condizione necessaria data la natura asincrona dell'intero backend. La modalità automatica configurata nel progetto rende superflua l'annotazione di ogni singolo test _{GG} .
pytest-cov	7.1.0	Misurazione della copertura del codice _G raggiunta dai test _{GG} del backend.
respx	0.23.1	Interposizione sulle richieste HTTP effettuate tramite httpx. Consente di verificare _G il comportamento del client verso il provider dei modelli — flusso corretto, risposta malformata, errore di connessione, scadenza del tempo massimo — senza contattare alcun servizio esterno, rendendo i test _{GG} deterministici e indipendenti dalla rete.

Alternative considerate

- **Jest al posto di Vitest**

Jest è lo standard storico dei test_{GG} in ambiente JavaScript, con la comunità più ampia e la maggiore disponibilità di documentazione. Richiederebbe però una configurazione separata e mantenuta in parallelo a quella di Vite, in particolare per la trasformazione di TypeScript e per i moduli ES, che Jest non supporta nativamente. Il disallineamento fra le due configurazioni è una fonte di errori ricorrente, e non è compensato da alcun vantaggio funzionale: l'interfaccia di programmazione di Vitest è deliberatamente compatibile con quella di Jest, e i test_{GG} risultano pressoché identici.

- **unittest al posto di pytest**

Il modulo `unittest` è incluso nella libreria standard di Python e non aggiunge dipendenze. Impone però una struttura a classi che rende i test_{GG} più verbosi e, soprattutto, non dispone né di un sistema_G di *fixture* componibili paragonabile a quello di `pytest` né dell'ecosistema di estensioni sul quale il progetto si appoggia per l'esecuzione asincrona, la misurazione della copertura del codice_G e l'interposizione sulle richieste HTTP.

2.6 Infrastruttura

Le tecnologie di questa sezione non concorrono al comportamento del prodotto in esecuzione, ma governano il modo in cui esso viene costruito, avviato, verificato_G e mantenuto coerente fra i due livelli.

Organizzazione della repository_G. Prima delle tecnologie va richiamata una decisione organizzativa che le condiziona tutte: il gruppo ha adottato un'unica repository_{GG} (*mono-repo*) per i due livelli del prodotto. Il codice del frontend_{GG} risiede in `/web` e quello del backend in `/api`, ciascuno con il proprio gestore di pacchetti, il proprio file di dipendenze e il proprio file di lock. Restano quindi due progetti distinti, con costruzioni e dipendenze proprie: condividono la repository_{GG} e la cronologia delle modifiche, non divengono un unico artefatto_{GG}.

La scelta ha due conseguenze dirette sulle tecnologie descritte di seguito. La prima è che una variazione del contratto dell'API_{GG} e il corrispondente adeguamento del client possono essere integrati_G con una sola pull request_{GG}, che li sottopone alla medesima verifica_{GG} automatica: non esiste un intervallo in cui i due livelli risultino disallineati. La seconda è che la configurazione di verifica_{GG} è definita una volta sola, in un solo insieme di flussi di lavoro.

Tabella 5: Tecnologie di infrastruttura

Tecnologia	Versione	Descrizione e ruolo nel progetto
Docker Docker Compose	29.4.3 5.1.4	Ambiente di sviluppo _{GG} ed esecuzione riproducibile. Il file <code>docker-compose.yml</code> descrive i due servizi <code>web</code> e <code>api</code> , la rete che li collega, le porte esposte verso la macchina ospite e le variabili d'ambiente, e li avvia con un solo comando. Le immagini di base sono fissate dai rispettivi <code>Dockerfile</code> : <code>node:24-alpine</code> per il frontend _{GG} e <code>python:3.12-slim</code> per il backend. La versione di Node.js è quella dell'ultima linea a supporto esteso: il perimetro tecnologico del frontend _{GG} (Vite 8, ESLint 10, TypeScript 6.0.3) dichiara vincoli sul motore che le linee precedenti non soddisfano più.
GitHub Actions	—	Infrastruttura di <i>Continuous Integration</i> _G . Su ogni pull request _{GG} verso il ramo principale esegue, separatamente per i due livelli, l'analisi statica _G , la verifica _{GG} dei tipi, la suite di test _{GG} con misurazione della copertura del codice _G e la costruzione del pacchetto di produzione. È il luogo in cui gli strumenti delle sezioni 2.5.1 e 2.5.2 vengono effettivamente applicati: senza di esso la loro esecuzione dipenderebbe dalla diligenza individuale. Trattandosi di un servizio della piattaforma e non di una dipendenza installata, non è associato a una versione dichiarata nei file di progetto; le versioni fissate sono quelle delle singole azioni richiamate dai flussi di lavoro.

Tecnologia	Versione	Descrizione e ruolo nel progetto
pnpm	11.4.0	Gestore dei pacchetti del frontend _{GG} . Memorizza ogni versione di ogni pacchetto una sola volta e la collega per riferimento, riducendo occupazione di spazio e tempi di installazione. Il file <code>pnpm-lock.yaml</code> fissa le versioni risolte dell'intero albero delle dipendenze; la versione dello strumento stesso è vincolata dal campo <code>packageManager</code> di <code>package.json</code> , così che sviluppo _{GG} locale e integrazione _{GG} continua adoperino il medesimo risolutore.
uv	0.11.16	Gestore dei pacchetti e dell'ambiente virtuale del backend. Risolve le dipendenze dichiarate in <code>pyproject.toml</code> e ne fissa le versioni in <code>uv.lock</code> ; l'installazione in fase di costruzione dell'immagine avviene in modalità vincolata al file di lock, così che l'ambiente sia identico su ogni macchina.
openapi-typescript	7.13.0	Generazione dei tipi TypeScript del client a partire dallo schema OpenAPI prodotto da FastAPI. Il frontend _{GG} non dichiara autonomamente la forma dei dati scambiati con il backend: la deriva dal contratto pubblicato dal backend stesso. Una modifica di uno schema Pydantic si propaga così ai tipi del client e, se incompatibile con l'uso che il frontend _{GG} ne fa, si manifesta come errore di compilazione anziché come guasto in esecuzione. Il file generato è escluso dall'analisi di ESLint, non essendo codice scritto a mano.

Nota sulle versioni degli strumenti di sviluppo. Le versioni indicate per **Docker**, **Docker Compose**, **pnpm** e **uv** vanno lette diversamente da tutte le altre del documento. Quelle delle librerie sono fissate dai file di lock e risultano identiche su ogni macchina; queste quattro no, perché nessun file del progetto le vincola: le immagini installano i due gestori di pacchetti senza indicare una versione, e Docker e Compose sono presupposti dall'ambiente anziché dichiarati. I valori riportati sono quindi quelli **in uso sull'ambiente di sviluppo del gruppo** alla data del documento, e non un vincolo_{GG} verificabile_G: su un'altra macchina, o in un altro momento, l'installazione può risolvere versioni diverse. Per Docker il numero è quello di CLI e motore riportato da `docker --version`, che non coincide con la versione di Docker Desktop, la quale segue una numerazione propria.

Verifica automatica delle pull request_G. I controlli eseguiti dall'integrazione_{GG} continua sono la condizione che ogni modifica deve soddisfare prima di essere integrata_G nel ramo principale. Sono organizzati in due processi indipendenti, uno per livello, ciascuno con i propri passi:

- **frontend_G**: installazione vincolata al file di lock, ESLint, compilazione dei tipi con `tsc`, esecuzione di Vitest con misurazione della copertura del codice_G e verifica_{GG} delle soglie, costruzione del pacchetto di produzione con Vite;
- **backend**: installazione vincolata a `uv.lock`, Ruff (regole e formattazione), mypy, esecuzione di pytest con `pytest-cov` e verifica_{GG} delle soglie.

L'esito negativo di un qualunque passo impedisce l'integrazione_{GG}. Questi controlli soddisfano il requisito_G **R-4-Q-Ob**, che subordina l'integrazione_{GG} nel ramo principale al superamento dei test_{GG} automatici oltre che alla revisione tra pari, e rendono **verificabile** quanto la voce `openapi-typescript` afferma: perché un'incompatibilità di contratto si manifesti “come errore di compilazione” occorre che una compilazione avvenga a ogni modifica, in un ambiente che non si possa aggirare. La rigenerazione dei tipi dallo schema OpenAPI è essa stessa un passo del

processo del frontend_{GG}, seguito dal controllo che il file generato coincida con quello versionato: se i due differiscono, lo schema del backend è cambiato senza che il client sia stato aggiornato, e la verifica_{GG} fallisce.

Nota sui container. Un container non è una macchina virtuale, bensì un normale processo del sistema_{GG} operativo ospite, isolato mediante funzionalità del kernel: i *namespaces* gli attribuiscono una vista propria di processi, rete e file system, i *cgroups* ne limitano il consumo di risorse. Poiché il kernel è condiviso con la macchina ospite, anziché replicato come in una macchina virtuale, l'ingombro è sensibilmente inferiore e l'avvio richiede secondi anziché minuti.

Alternative considerate

- **Configurazione manuale dell'ambiente al posto di Docker**

Richiederebbe a ciascun membro del gruppo di installare e allineare manualmente le versioni di Node.js, di Python e delle rispettive dipendenze di sistema_{GG} su tre famiglie di sistemi_G operativi differenti. Le configurazioni divergerebbero inevitabilmente, con la conseguenza che un comportamento osservato su una macchina non sarebbe riproducibile sulle altre — circostanza che rende la diagnosi dei guasti sproporzionatamente onerosa e vanifica il valore probatorio dei test_{GG}.

- **Podman al posto di Docker**

Podman è compatibile con i formati e i comandi di Docker e presenta un vantaggio di sicurezza_G non trascurabile: non richiede un processo demone privilegiato in esecuzione permanente e consente di eseguire i container senza privilegi amministrativi. È stato scartato per ragioni di diffusione, ampiezza della documentazione e familiarità pregressa dei membri del gruppo, elementi che riducono il tempo speso in attività non produttive.

- **Poly-repo al posto del mono-repo**

La separazione di frontend_{GG} e backend in repository_{GG} distinte è appropriata quando i due componenti sono sviluppati da gruppi diversi e rilasciati in modo indipendente. Non è il caso del progetto: le due parti sono sviluppate dallo stesso gruppo e rilasciate congiuntamente. La separazione imporrebbe due pull request_{GG} coordinate a ogni variazione del contratto dell'API_{GG}, con una finestra temporale in cui le due repository_{GG} risultano disallineate, oltre alla duplicazione della configurazione di verifica_{GG}.

- **Tipi TypeScript scritti a mano al posto della generazione da OpenAPI**

È l'alternativa che non aggiunge alcuno strumento, ma introduce una duplicazione del contratto dell'API_{GG} in due punti del sistema_{GG} mantenuti separatamente. Le due definizioni divergono inevitabilmente non appena il backend evolve senza che il frontend_{GG} venga aggiornato di conseguenza; poiché i tipi TypeScript sono rimossi in compilazione, la divergenza non produce alcun errore e si manifesta come guasto silenzioso in esecuzione. La generazione a partire dallo schema elimina la duplicazione alla radice.

3 Architettura

3.1 Vista d'insieme

3.2 Architettura di deployment

3.2.1 Confronto con l'architettura a microservizi

3.2.2 Confronto con l'architettura monolitica

3.3 Architettura del frontend

3.3.1 Scomposizione in componenti

3.3.2 Gestione dello stato applicativo

3.4 Architettura del backend

3.4.1 Porte e adattatori

3.4.2 Scomposizione in moduli

3.5 Design pattern adottati

3.6 Idiomi e convenzioni di codifica

3.6.1 Generatori asincroni e propagazione dell'annullamento

3.6.2 Tipizzazione dei confini fra i livelli

3.6.3 Denominazione, struttura dei file e gestione degli errori

4 Flusso dei dati

4.1 Generazione assistita da LLM_G con risposta in streaming

4.2 Annullamento di un'operazione in corso

4.3 Generazione a partire da un collegamento

4.4 Salvataggio e caricamento di una nota

4.5 Propagazione e presentazione degli errori

5 Tracciamento dei requisiti $_G$

5.1 Criteri di tracciamento

5.2 Requisiti $_G$ funzionali

5.3 Requisiti $_G$ di qualità

5.4 Requisiti $_G$ di vincolo $_{GG}$

5.5 Grafici riassuntivi